SC2002 FCS1 Group 4 Assignment

Main Github Page: https://github.com/T4ST13/SC2002-FCS1-Grp-4

1. Introduction

1.1 Objective

This project is a command-line, turn-based combat system designed to demonstrate object-oriented design. The primary objective is to model a flexible battle engine that allows new actions, status effects, items, and characters to be added with minimal modification. Players can choose Warrior or Wizard to battle against Goblin and Wolf. The game supports actions, items, status effects, turn order, difficulty levels, and backup spawns.

1.2 Scope

The system supports:

- Controlled characters (Warrior, Wizard)
- Enemy characters (Goblin, Wolf)
- Turn-based combat with speed ordering
- Actions: attack, defend, skills, and items
- Status effects with multiple turn durations
- Difficulty levels to choose from and enemy spawning

1.3 System Overview/ Key Requirements

The system is structured into three logical layers of architecture, ensuring separation of concerns and improving maintainability:

1. Presentation Layer: ConsoleUI (user inputs/outputs)
2. Control Layer: BattleEngine (manages game structure)
3. Domain Layer: Combatants, Actions, StatusEffects

2. System Architecture

2.1 Layered Design

The system follows a layered architecture:

- UI layer - Player interactions (ConsoleUI)
- Control layer - Coordinates game logic and turn flow (BattleEngine)
- Domain layer - Contains the game entities and logic

2.2 Key Components

Component	**Role**
Action	Trigger - executable action
ActionLogic	Behaviour of actions
StatusEffect	Ongoing Logic - stores effect data
StatusEffectLogic	Effect behaviour
SkillLogic	Cooldown Control
ItemLogic	Special Actions
Level	Setup - Handles difficulty and enemy spawning
Combatant	Player/enemy stats

3. UML Class Diagram

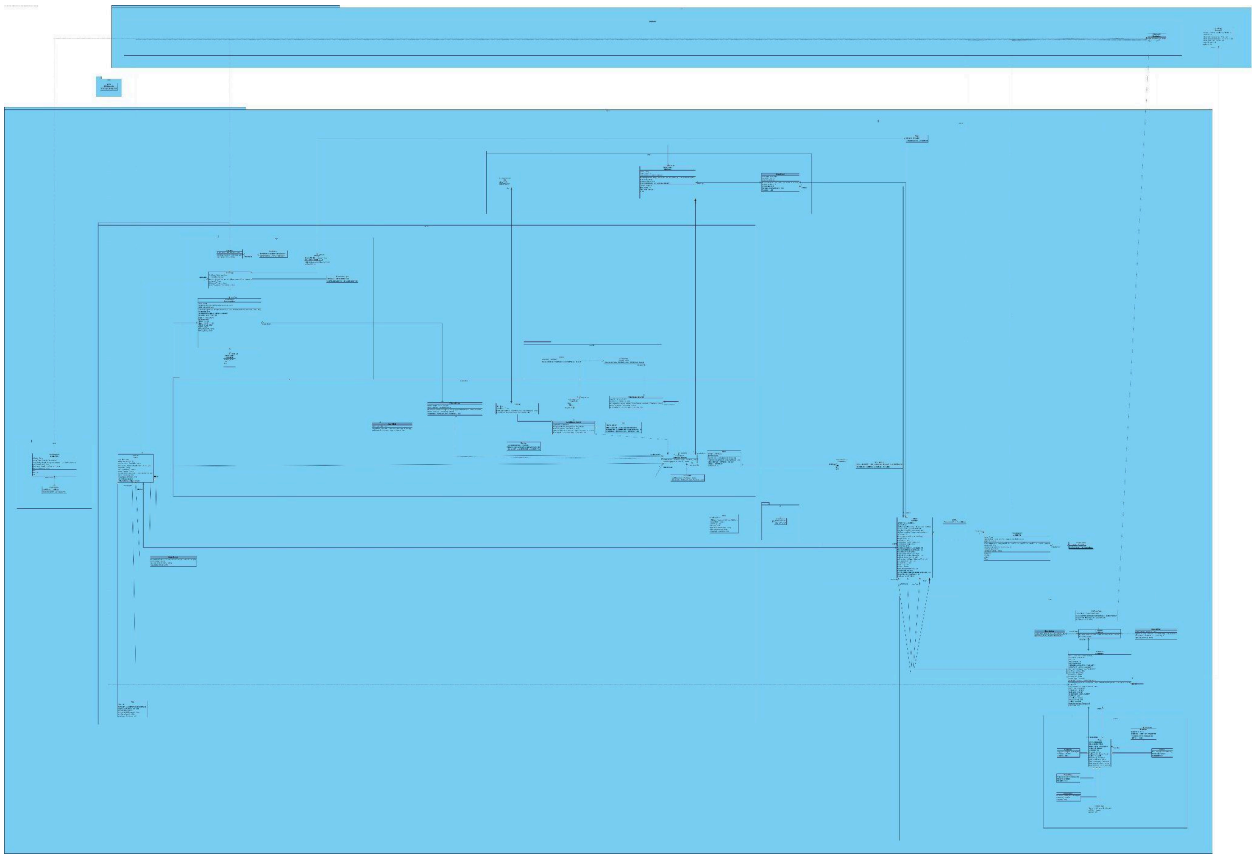


Figure 1: UML Class Diagram

Click on image for link to separate drive file (to zoom)

The UML diagram illustrates key relationships:

- Inheritance: Combatant → Player/Enemy, Action → ShareAction/Skill/Item
- Composition: Combatant owns its Action list and StatusEffect list.

The design also applies a strategy-style abstraction through TurnDecider, where different round-order policies such as SpdBasedOrder and HpBasedOrder can be used without changing the engine. This structure reflects the modular layering of UI, control, and domain components in the final system.

This structure ensures extensibility and modularity. The overall structure of the system and the relationships between its core components follow a modular and layered architecture, consisting of boundary, control, and domain layers.

3.1 System structure

The system can be divided into three main layers:

- Boundary Layer: UI class displays messages, shows available options, and handles user input; it serves as the interface between the user and the system.
- Control Layer: BattleEngine class manages battle loop, enemy wave loading, manages turn execution, and phase progression.
- Domain Layer: Contains the core game entities such as Combatant, Action, StatusEffect, ActionLogic, EffectLogic, and associated logic classes. This layer encapsulates the main functionality of the combat system.

This layered separation improves maintainability by ensuring that user interaction, control logic, and core game mechanics are decoupled.

3.2 Combatant Hierarchy

The combat hierarchy is centered on the abstract Combatant class. It stores the shared battle state and behaviour common to all combat participants, such as combat type, current HP, combat statistics, and action meter. It also provides common methods for turn setup, stat access, status effect activation, HP updates, and action execution.

Combatant is extended by two subclasses (Player and Enemy).

- Player: Player-only actions to base class (e.g. defend and class skill)
- Enemy: Reuses shared combat behaviour from Combatant without adding player-only functions (only basic attacks)

Concrete characters (e.g. Warrior, Goblin) are represented by CombatType enum values rather than separate subclasses. These enum values act as a template for each combatant's fixed base stats, while the inheritance hierarchy still separates player-only behaviour from enemy behaviour. This design maintains code reuse and polymorphism, as the battle engine can still treat all battle participants through common Combatant type.

3.3 Action System Design

The action system is designed using a composition-based approach. Each Action object contains a reference to an ActionLogic object, while the exact action behavior is determined by selected ActionLogicType. The main concrete Action subclasses are SharedAction, Skill, and Item, while the main logic wrappers are SharedActionLogic, SkillLogic, and ItemLogic.

Instead of creating a separate concrete class for every individual action such as Basic Attack or Defend, the current design stores these behaviours inside the ActionLogicType enum. Each enum constant contains a list of CombatMechanism objects (e.g. Damage, Heal, ApplyEffect, or ConditionalMechanism), which are executed in sequence when the action is used. For example, Basic Attack is represented by BASIC_ATTACK_LOGIC, Defend by DEFEND_LOGIC, and the different items and skills are represented by their own enum values. This makes the action system more modular, because complex action behaviour is built by combining reusable mechanism objects rather than hardcoding everything into the battle engine.

3.4 Status Effect System

Focused on concrete StatusEffect class and enum EffectLogic. A StatusEffect object stores three key pieces of data:

- Selected Effect Logic
- Remaining Turn Count
- Combatant

The effect behaviour itself is not implemented through separate subclasses such as StunEffect or DefendEffect. Instead, each effect is represented by an EffectLogic enum value, such as DEFEND_EFFECT, SMK_BOMB_INV, ARC_BLAST_EFFECT, or STUN.

Each EffectLogic value contains a base duration and a list of CombatMechanism objects that define what the effect does when triggered. For example, Defend applies a defence buff, Smoke Bomb adds an invulnerability flag, Arcane Blast applies an attack buff, and Stun adds a no-action flag. This design keeps the data for effect duration and type inside StatusEffect, while the reusable effect behaviour is configured through EffectLogic and the mechanism classes.

3.5 Turn-Based Mechanism

The TurnBasedCount interface manages both the skill cooldowns and the status effect duration in the system. This interface is implemented by classes such as Skill and StatusEffect, through the shared method passTurn(). This shared interface allows the program to update skill cooldowns and status-effect durations using the same abstraction, even though they belong to different parts of the system.

This logic is coordinated inside Combatant.finishTurn(). At the end of a combatant's turn, the method iterates through active status effects and action objects. Status effects reduce their remaining duration through passTurn(), while any action object that also implements TurnBasedCount (e.g. Skill) has its cooldown reduced in the same way.

This is a good example of polymorphism, because the system can treat both cooldowns and timed effects uniformly through a shared interface.

3.6 Class Relationships

Several key relationships are used:

- Inheritance (Generalisation):
    - Combatant hierarchy: Player, Enemy
    - Action hierarchy: SharedAction, Skill, Item
    - ActionLogic hierarchy: SharedActionLogic, SkillLogic, ItemLogic
- Composition: Combatant owns its list of Action objects and active StatusEffect objects
- Association: BattleEngine interaction with Combatant during turn execution. ActionLogicType and EffectLogic also uses enum-based configuration to define reusable behaviour through CombatMechanism composition

These relationships work together to contribute to a modular and well-structured system.

3.7 Design Patterns

The current design combines 3 main design ideas. First, action and effect behaviour are built compositionally and reusable CombatMechanism objects such as Damage, Heal, and Buff.

Second, the battle loop is also organised into a phase structure using the Phase hierarchy and PhaseFlow. The phases (TriggerEffect, ChooseAction, ChooseTarget, ApplyAction, and EndTurn) break a turn into clear steps, improving readability and separating responsibilities inside the engine.

Third, turn ordering is abstracted using the TurnDecider interface. This allows different round-order policies such as SpdBasedOrder and HpBasedOrder to be used without changing the engine.

4. Design Considerations & OOP Concepts

4.1 Important Design Decisions:

Separation of Concerns

Each system (actions, effects, items, level) is separated into different packages. This helps to improve the readability and maintainability. Each component handles a specific aspect of the game, reducing complexity and improving maintainability. This benefits the project when changes in one subsystem do not impact others.

- Actions (Action, ActionLogic) → execution
- Status Effects(StatusEffect, EffectLogic) → ongoing logic
- Items(ItemLogic) → ongoing logic
- Engine → control flow
- LevelConfig
- CombatMechanism → Single block of mechanism

Extensibility and Scalability

Supported through configuration-based design. New action or effect behaviour can be assembled by combining reusable mechanisms inside ActionLogicType and EffectLogic, while new turn-order policies can be added through the TurnDecider abstraction. New mechanisms can be easily created as well by creating a new class that implements the CombatMechanism interface.

This reduces duplicated logic and keeps the engine insulated from low-level combat behaviour.

Future features, such as new items, can be added with minimal effort.

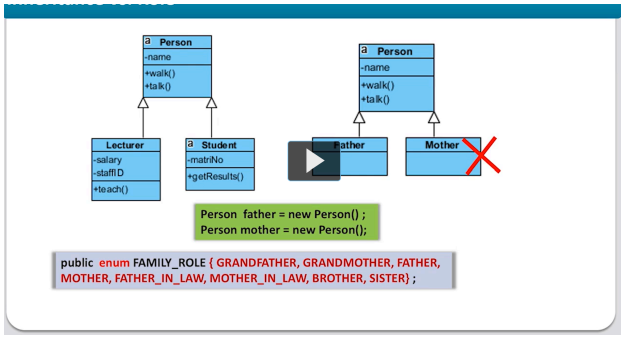
Reusability

- Common logic used is placed in base classes to avoid duplication across multiple actions and effects, such as:
    - ActionLogic
    - EffectLogic
    - CombatMechanism

Flexibility

- By treating items and skills as types of actions, the system maintains a consistent execution model. It is easy to add new characters or items without changing existing code, which adheres to the Liskov Substitution Principle. Similarly, each class implementing the CombatMechanism represents a fairly specialized mechanism, meaning that they can be joined in multiple combinations to create items or skills with unique and diverse logic.

Subclasses vs Enums

- We initially considered creating separate subclasses for each Combatant type, skills, and items. However, we did not have a clear justification for doing so, as none of the subclasses would have attributes or methods compared to its super class. For example, creating separate Wizard and Warrior classes under the Player class would only require assigning different values to the existing stat attributes. Similarly, in our design, a Skill object holds a list of CombatMechanism objects, and only a single function that iterates through and activates each mechanism object is required for any type of an Item or Skill. This directly goes against the Design principles mentioned in the Wk9 lecture videos, which mentions that there should be a clear justification such as new attributes or overridden methods to create a subclass. Hence, we instead decided to use enumerations to adhere to this requirement.
- 

5. OOP Concepts

Polymorphism

Polymorphism allows objects of different classes to be treated through a common parent type. In our project, this is used in the action system, combatant entities, and status effect system.

This design makes the system easier to extend. New actions or effects can be added as new subclasses without changing the overall flow or code.

Example:

TurnBasedCount is a shared interface used for objects whose values decrease over turns. It's implemented by both Skill and StatusEffect, even though they represent different parts of the system. This allows both cooldown and effect duration to be handled through the same shared interface, while each class provides their own implementation of methods (e.g. passTurn()).

```
public class Skill extends Action implements TurnBasedCount{
    }

    public void passTurn(){//for decreasing skill cd when turn passes
        if (remainingCd > 0) {//disable negative cooldowns
            remainingCd--;
        }
    }
```

Figure 2: TurnBasedCount Interface implemented by Skill.java

Usage:

Another example is TurnDecider, where the battle engine can work with either SpdBasedOrder or HpBasedOrder through the same abstraction without changing its own logic.

Inheritance

Inheritance allows a subclass to reuse attributes and methods from a parent class. This is used to model shared battle behaviour while allowing each concrete class to keep its own identity, with its own attributes. By using inheritance, the project avoids duplicate code, and keeps related classes organised under clear parent abstractions.

Example:

Combatant is the main abstract superclass for battle participants. It stores shared attributes such as combatType, currentHP, actionList, and actionMeter. It also provides common methods such as getBaseStat(), addAction(), and takeDamage().

Player and Enemy both inherit from Combatant. This means both sides in battle automatically reuse the same HP and stats handling logic. Player then extends this shared base by adding Defend any player-specific skills through its constructor, while Enemy uses inherited behaviour directly, keeping enemies simpler.

```java
// Shared by both Player & Enemy
public abstract class Combatant implements Displayable, Comparable<Combatant> {
    private final CombatType combatType;
    private int currentHP;
    private Map<CombatStat, Integer> statMap = new HashMap();//stats after buffs
    private List<Action> actionList = new ArrayList<>();;
    private List<StatusEffect> effectList = new ArrayList<>();;
    private Map<Flag, String> flagMap = new HashMap<>();//Map flag to name of the effect that created it
    private int actionMeter;

    /* == Constructor for combatant == */
    protected Combatant (CombatType combatType) {
        this.combatType = combatType;
        addAction(new SharedAction(this, new SharedActionLogic(ActionLogicType.BASIC_ATTACK_LOGIC)));//all subclasses of combatants can use basic attack
        //might change new Logic to getInstance
        this.currentHP = this.combatType.getBaseStat(CombatStat.HP);
        this.setupForTurn();
        this.actionMeter = this.combatType.getBaseStat(CombatStat.SPD);
    }

    public int getCurrentHP(){
        return currentHP;
    }

    public CombatType getCombatType(){
        return combatType;
    }
```

Figure 3: Shared fields in Combatant

```java
public class Player extends Combatant {

    private final SkillLogic skillLogic;

    public Player(CombatType playerType) {
        super(playerType);
        this.addAction(new SharedAction(this, new SharedActionLogic(ActionLogicType.DEFEND_LOGIC)));//all playable characters can defend
        //change 'new Logic()' to getInstance()
        this.skillLogic = SkillLogicMap.getSkillLogic(playerType);
        this.addAction(new Skill(this, this.skillLogic));
    }

    public SkillLogic getSkillLogic(){
        return skillLogic;
    }
}
```

Figure 4: Player-specific methods in Player.java

Usage:

By extending shared parent classes, the project avoids duplicating battle logic across multiple files. For example, all combatants automatically inherit the same damage and turn-related behaviour from Combatant, while Player adds what is unique for player characters.

Encapsulation

Encapsulation protects class data by keeping important fields inside the class and only allows controlled access through methods. In our project, this is used to protect combat state and reduce invalid updates. This improves reliability, because class state is only changed through valid operations, instead of being edited freely outside the class.

Example:

In Combatant, important data such as combatType, currentHP, and other stats are declared privately within the class, so other classes cannot modify them directly. These fields are accessed through getter methods such as getName(). For example, takeDamage() ensures HP does not go below 0, while heal() ensures HP does not exceed combatant's base maximum.

Encapsulation is also shown through controlled access using protected constructors, such as in Combatant (protected), so only Player and Enemy can initialise shared combatant state directly. This limits object creation to intended class hierarchy instead of being freely accessible to unrelated classes.

```java
public abstract class Combatant implements Displayable, Comparable<Combatant> {

    public String getName() {
        return combatType.getName();
    }

    public int getBaseStat(CombatStat statType){
        return combatType.getBaseStat(statType);
    }

    public int getFinalStat(CombatStat statType){
        return statMap.get(statType);
    }
```

Figure 5: Methods in Combatant.java

Usage:

This makes the code more reliable because important state changes are handled through specific methods instead of being edited directly from outside the class. For instance, battle logic does not directly modify a combatant's HP, but instead calls takeDamage() or heal(), so the state is always valid.

Abstraction

Abstraction identifies common features of a group of objects and represents shared behaviour through general types. This is used in our project to model combatants, actions, and status effects in a general way.

Example:

Action abstracts all executable player or enemy actions. ActionLogic is an abstraction that connects an Action object to its configured behaviour, while most detailed behaviour is defined through ActionLogicType.

An important abstraction is the CombatMechanism interface. Different mechanisms such as Damage, Heal, ApplyEffect, Buff, and ConditionalMechanism all implement the same logic, but each performs a different type of combat behaviour. Because actions and effects are built from these shared abstractions, the program can define different behaviours in a modular and reusable way. This improves extensibility and reduces duplication across the system.

```java
package domain.combatmechanism;

import domain.combatant.Combatant;

public interface CombatMechanism {
    public /*String*/ void execute(Combatant user, Combatant target);

    public abstract boolean needsTarget(Combatant user);
}
```

Figure 6: CombatMechanism.java Interface

Usage:

Higher-level logic can work with general types like Combatant and Action instead of depending only on concrete classes. Shared abstractions improves the extensibility of the program for future additions, and avoids duplication of codes.

Another Example:

Another example of abstraction would be the Phase object. Each subclass of the abstract Phase class realizes the abstract methods canEnter() and execute() according to their needs, as different battle phases have different requirements to enter or run. However, the battle engine only recognizes them as a type of Phase, and simply runs the canCheck() and execute() methods to determine which phase should be entered, without having to know the exact implementations or details regarding a specific phase.

6. SOLID Principles

This system applies SOLID principles to ensure scalability, maintainability, and clean architecture. Each principle is demonstrated below:

S - (Single Responsibility Principle)

- Each class has one single responsibility:
    - ActionLogic - defines action behavior, handles action execution only
    - StatusEffectLogic - handles effects behaviour over time
    - Level - controls enemy generation (EASY, MEDIUM, HARD)
    - BattleEngine - manage turn flow and game state

Each class in the system is designed to have a single, well-defined responsibility, ensuring that changes in one part of the system do not affect unrelated components. For instance, BattleEngine is responsible only for managing the battle flow, including turn order, phase execution, and game state control. It does not implement combat logic such as damage calculation or status effects. These responsibilities are delegated to each of their own components, such as ActionLogic and CombatMechanism.

In addition, Combatant is also responsible for maintaining combat-related state, such as HP, stats, actions, and status effects, with no responsibility on how actions are executed. This separation prevents classes from becoming overly complex and improves maintainability, as modifications to combat behaviour or effects can be made without affecting the BattleEngine.

For examples:

```java
public class BattleEngine {//stores and manages overall flow of the battle
    private ConsoleUI mainUI = new ConsoleUI();

    private TurnDecider turnDecider;//holds logic for deciding turn order

    private int round;

    private int charOrderIndex;//indicates which char's turn it is

    private Combatant player;
    private List<Combatant> enemyList = new ArrayList<>();
```

Figure 7: BattleEngine.java

- In Figure 7, this shows that BattleEngine focuses purely on:
    - Turn order
    - Game state
    - Flow control

This does not implement damage, skills, or effects logic, which are handled elsewhere. Similarly, as seen from Figure 8, the Combatant is responsible only for holding the combat-related state and shared behaviour.

```java
public abstract class Combatant implements Displayable, Comparable<Combatant> {
    private final CombatType combatType;
    private int currentHP;
    private Map<CombatStat, Integer> statMap = new HashMap();//stats after buffs
    private List<Action> actionList = new ArrayList<>();;
    private List<StatusEffect> effectList = new ArrayList<>();;
```

Figure 8: Combatant.java

- This separation ensures:
    - Battle flow, combat logic, and effect logic are not equal to one another.
    - Changes in one area do not affect the others

O - (Open/Closed Principle)

- The System is designed to be open for extension but closed for modification, particularly in the action and effect systems:
    - New actions can be added by creating new subclasses of ActionLogic.
    - New status effects can be added by extending StatusEffectLogic

- No need to modify existing code.

Instead of hardcoding behaviours into the engine, actions are defined through the combination of ActionLogic, ActionLogicType using (enum), and CombatMechanism (objects). New actions are introduced by adding new ActionLogicType values and configuring them with appropriate CombatMechanism implementations, such as Damage, Heal, or ApplyEffect. This allows the new behaviours to be added without modifying other existing classes, such as BattleEngine or Combatant. Additionally, New effects can also be added through EffectLogic without changing existing logic. This design enables the system to scale easily while reducing the risk of introducing bugs into existing functionality.

For examples:

```java
public abstract class ActionLogic{//holds logic for an action

    ActionLogicType logicType;
    private final boolean consumeTurn;

    protected ActionLogic(ActionLogicType logicType, boolean consumeTurn) {/
        this.logicType = logicType;
        this.consumeTurn = consumeTurn;
    }
```

Figure 9: ActionLogic.java

The abstraction ActionLogic Figure 9 allows new behaviours without modifying existing code. New behaviours are introduced through ActionLogicType, as seen in Figure 10.

```java
    public enum ActionLogicType {//holds defined mechanisms of each action logic
        //Shared Action Logic
        BASIC_ATTACK_LOGIC(
            "Basic Attack",
            Arrays.asList(
                new Damage(TargetMode.TARGET)//might take maxTarget out
            ),
            ActionType.SHARED_LOGIC
        ),
        DEFEND_LOGIC(
            "Defend",
            Arrays.asList(
                new ApplyEffect(TargetMode.SELF, EffectLogic.DEFEND_EFFECT)
            ),
            ActionType.SHARED_LOGIC
        ),

        //Item Logic
        POTION_LOGIC(
            "Potion",
            Arrays.asList(
                new Heal(TargetMode.SELF, 10)
            ),
            ActionType.ITEM
        ),
        POWER_STONE_LOGIC(
            "Power Stone",
            Arrays.asList(
                new ActivateSkill()
            ),
            ActionType.ITEM
```

Figure 10: ActionLogicType.java

As the behaviour is defined via:

- ActionLogic
- ActionLogicType
- CombatMechanism composition

New actions can be added by adding a new enum value and defining its mechanisms. No changes required in BattleEngine or Combatant, and this satisfies OCP by extending functionality without modifying existing classes.

L - (Liskov Substitution Principle)

- Subclasses (Player - Warrior and Wizard, Enemy - Goblin and Wolf) can all be used whenever Combatant can be used interchangeably whenever a Combatant is expected, without affecting correctness. From Figure 11, the engine does not care about the actual type.

The BattleEngine operates on the abstract type Combatant when managing turn order and executing actions. It does not require knowledge to know if the object is a player or an enemy. Additionally, all subclasses preserve expected behaviour, such as taking damage, applying status effects, or executing actions. This ensures that substituting one subtype for another does not break the system logic or introduce bugs and inconsistencies into the gameplay, and the system remains flexible and easy to extend with new combatant types.

For examples:

```
        private Combatant player;
        private List<Combatant> enemyList = new ArrayList<>();
        private List<List<Combatant>> enemyWaves;
        private List<Combatant> charOrder = new ArrayList<>();
```

Figure 11: BattleEngine.java

```
public abstract class Combatant implements Displayable, Comparable<Combatant> {
    private final CombatType combatType;
    private int currentHP;
    private Map<CombatStat, Integer> statMap = new HashMap();//stats after buffs
    private List<Action> actionList = new ArrayList<>();;
    private List<StatusEffect> effectList = new ArrayList<>();;
    private Map<Flag, String> flagMap = new HashMap<>();//Map flag to name of the effect that created it
    private int actionMeter;
```

Figure 12: Combatant.java

- Combatant is the base abstraction, and it is used generically in BattleEngine
- The battle system does not need to know the exact type. For example, we can do Combatant c = new Warrior();. The engine treats all combatants uniformly, ensuring that no special case logic for subclasses and behaviour remains consistent.

With LSP, the BattleEngine can process all combatants uniformly, simplifying logic and reducing coupling. This prevents the systems from being fragile and harder to extend.

I - (Interface Segregation Principle)

The system uses small and focuses interfaces so that classes only depend on the methods they actually use. For example, only Player classes interact with skill-related methods.

- Player contains skill-related methods (cooldown, special skill)
- Enemy does not implement unnecessary methods. For example, Enemy do not have skill cooldown logic or special skill methods.

For example, in Figure 13, the TurnBasedCount interface defines the method passTurn( ). This interface is implemented only by classes that require turn-based updates, such as Skill for cooldown reduction and StatusEffect for duration tracking. In Figure 14, classes such as Combatant or Action do not require this behaviour and are not forced to implement unnecessary methods. This prevents unnecessary dependencies and keep the classes lightweight, improving clarity and maintainability.

```java
//for actions that change state every turn (ex. skill cooldown, status remaining turns)
public interface TurnBasedCount {
    void passTurn();
}
```

Figure 13: TurnBasedCount.java

- This interface is implemented only by classes that require turn-based updates, such as Skill and StatusEffect.

```java
    public void activateEffects() {
        for (StatusEffect effect : effectList){
            effect.execute(this);
            //the effect object already holds its user/creator as its class attribute, so we only pass the target
        }
    }
```

Figure 14: Combatant.java

- In Figure 14, this design ensures that no unnecessary methods are forced onto unrelated classes. Thus, classes remain lightweight and focused.
- This helps to prevent the combatant from forcefully implementing irrelevant methods, leading to poor design.

D - (Dependency Inversion Principle)

High-level modules (BattleEngine) depend on abstractions (Action, ActionLogic) rather than concrete class implementations, reducing coupling between system components.

For examples:

```
import domain.action.Action;
import domain.entity.Combatant;
```

Figure 15: BattleEngine.java

- Figure 15, BattleEngine interacts with abstract types such as Action(abstraction) and Combatant(abstraction), rather than specific concrete implementations like Player, Enemy, Skill, or Item.

```
private List<Action> actionList = new ArrayList<>();;
```

Figure 16: Combatant.java

- Figure 16, Combatant stores action as a list of abstract type Action. This allows different types of actions, such as Skill, Item, and SharedAction, to be used interchangeably, without modifying the Combatant.

```
import domain.action.logic.ActionLogicType;
import domain.action.logic.ItemLogic;
import domain.action.logic.SharedActionLogic;
```

Figure 17: Combatant.java

- The system also uses abstraction in logic handling. This helps to ensure that high-level systems, such as BattleEngine and the Combatant, do not depend on specific implementation. As a result, new logic types can be added without modifying core modules, and the system remains flexible and extensible.

Interaction between SOLID principles:

The SOLID principles work together in this system:

- SRP + OCP - Promotes modular and extensible design components.
- LSP + DIP - Ensures a flexible and decoupled system between modules.
- ISP - Keep class design clean and efficient

Summary of SOLID application:

- SRP: Classes, such as BattleEngine and Combatant, have clearly defined

responsibilities.

- OCP: New actions and effects are added through ActionLogicType without modifying existing code
- LSP: All subclasses of Combatant are used interchangeably
- ISP: interfaces like TurnBasedCount are minimal and specific
- DIP: Core systems depend on abstractions such as Action and Combatant

While the system follows the SOLID principles well, the use of enums such as ActionLogicType and EffectLogic helps to centralise the behaviour definitions. As the number of actions or effects increases, these enums may become large and harder to manage, and this represents a trade-off between simplicity and scalability, which may require refactoring into more dynamic structures in larger systems. Overall, these SOLID principles ensure that the system remains scalable, maintainable, and adaptable to future enhancements, for instance, new character classes, actions, or game mechanics.

7. Sequence Diagram

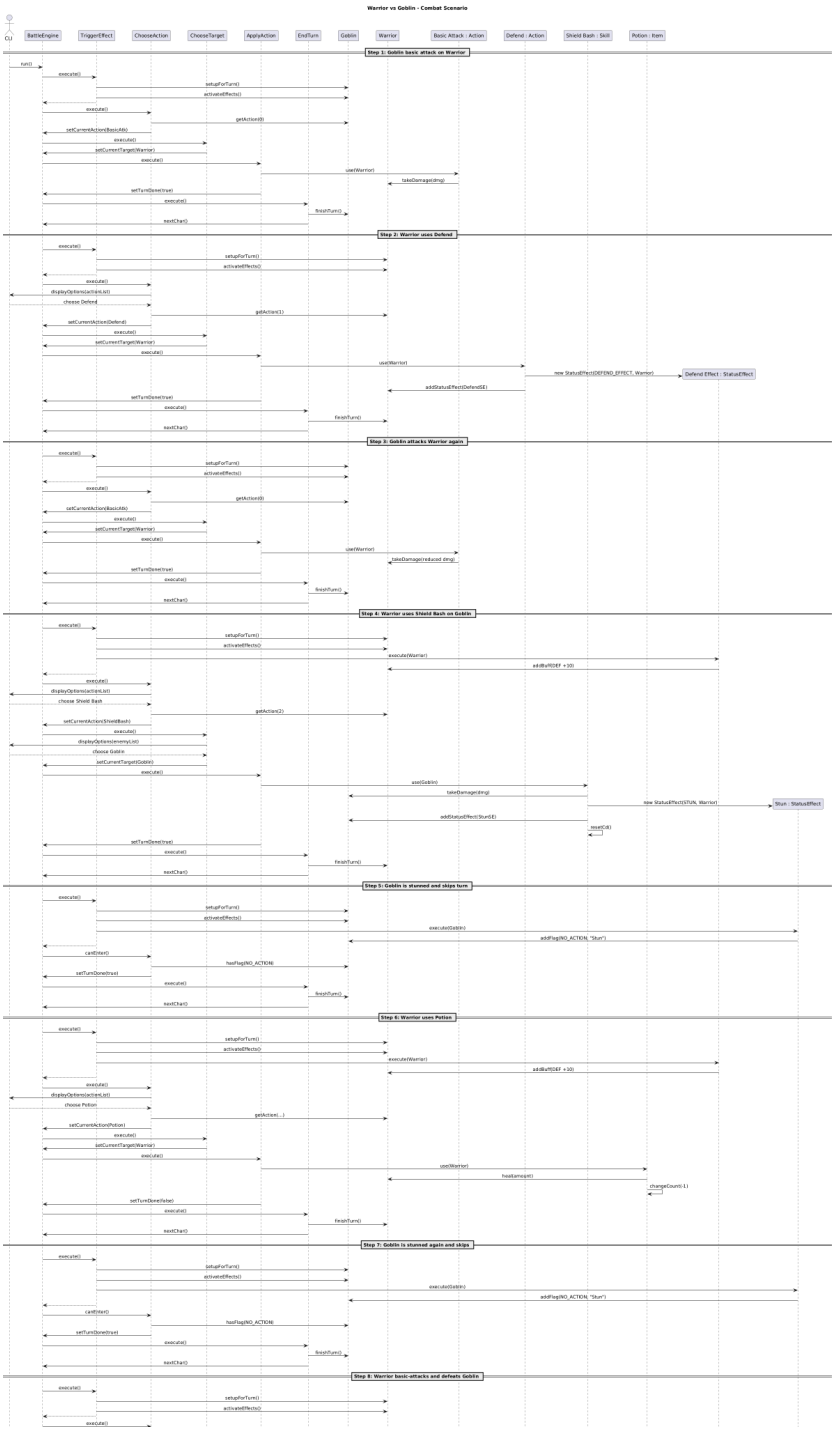


Figure 16: UML Sequence Diagram

Click on image for link to separate drive file (to zoom)

This sequence diagram illustrates a combat scenario between a Warrior and a Goblin. The process starts from the CLI boundary object. It transfers the input to BattleEngine. Then BattleEngine coordinates the turn flow, executes corresponding actions, updates HP and status effects, and monitors win/loss conditions. Entities named Warrior, Goblin, Action, and Item.

Boundary class: CLI

Entity classes: Combatant, Player, Enemy, Action, Item, Skill, StatusEffect, and various Logic classes (ActionLogic, SkillLogic, ItemLogic, and their respective subclasses).

For this project, we followed these steps:

1. Enemy's turn: Goblin executes basic attack on Warrior
2. Player's turn: Warrior defends against Goblin's next attack
3. Enemy's turn: Goblin executes basic attack, causing lower damage to Warrior
4. Player's turn: Warrior executes Shield Bash on Goblin, stunning it
5. Enemy's turn: Goblin is stunned, skips turn
6. Player's turn: Warrior uses Potion to heal
7. Enemy's turn: Goblin is stunned, skips turn
8. Player's turn: Warrior executes basic attack, eliminating Goblin

Step 1 - Goblin's Normal Attack

External Trigger: Goblin’s turn begins.

TriggerEffect: Goblin resets temporary turn state and activates any current status effects.

ChooseAction: Goblin automatically selects Basic Attack.

ChooseTarget: The target is set to Warrior.

ApplyAction: The selected Action is executed through currentAction.use(Warrior)

Internally: Damage.execute(Goblin, Warrior) is called → damage is calculated using Goblin’s ATK and Warrior’s DEF → Warrior.takeDamage(damage) is called.

Result: Warrior’s HP is updated.

EndTurn: Goblin finishes its turn, and turn-based values are updated.

Step 2 - Warrior's Defense

External Trigger: Warrior’s turn begins.

TriggerEffect: Warrior resets temporary turn state and activates any current status effects.

ChooseAction: The player selects Defend.

ChooseTarget: Defend does not require an enemy target, so the target is set to Warrior itself.

ApplyAction: The selected Action is executed through currentAction.use(Warrior).

Internally: ApplyEffect.execute(Warrior, Warrior) is called → Warrior.addStatusEffect(new StatusEffect(DEFEND_EFFECT, Warrior)).

Result: The Defend effect is added to Warrior.

EndTurn: Warrior finishes its turn, and turn-based values are updated.

Step 3 - Goblin attacks (with reduced damage).

External Trigger: Goblin’s turn begins again.

TriggerEffect: Goblin resets temporary turn state and activates any current status effects.

ChooseAction: Goblin automatically selects Basic Attack.

ChooseTarget: The target is set to Warrior.

ApplyAction: The selected Action is executed through currentAction.use(Warrior).

Internally: Damage.execute(Goblin, Warrior) is called → damage is calculated using Goblin’s ATK and Warrior’s current DEF → because Warrior has the Defend effect active, Warrior’s DEF is higher → Warrior.takeDamage(damage) is called.

Result: Warrior takes less damage than before, and Warrior’s HP is updated.

EndTurn: Goblin finishes its turn

Step 4 - Warrior uses Shield Bash

External Trigger: Warrior’s turn begins.

TriggerEffect: Warrior resets temporary turn state and activates its current status effects.

ChooseAction: The player selects Shield Bash.

ChooseTarget: The player selects Goblin as the target.

ApplyAction: The selected Skill is executed through currentAction.use(Goblin).

Internally: First, Damage.execute(Warrior, Goblin) is called → Goblin takes damage.

Then, ApplyEffect.execute(Warrior, Goblin) is called → Goblin.addStatusEffect(new StatusEffect(STUN, Warrior)). After the skill is used, the skill cooldown is reset by the Skill object.

Result: Goblin takes damage and receives the Stun effect.

EndTurn: Warrior finishes its turn, and cooldown / effect durations are updated.

Step 5 - Goblin skips its turn due to being stunned.

External Trigger: Goblin's turn begins.

TriggerEffect: Goblin resets temporary turn state and activates its current status effects.

Internally: The Stun effect executes and adds the NO_ACTION flag to Goblin for this turn.

ChooseAction: The engine checks whether Goblin can act. Since Goblin has the NO_ACTION flag, the action-selection phase is skipped.

Result: Goblin cannot act due to Stun.

EndTurn: Goblin finishes its turn, and the Stun duration is reduced.

Step 6 - Warrior uses a potion

External Trigger: Warrior's turn begins.

TriggerEffect: Warrior resets temporary turn state and activates any current status effects.

ChooseAction: The player selects Potion.

ChooseTarget: Potion does not require an enemy target, so the target is set to Warrior itself.

ApplyAction: The selected Item is executed through currentAction.use(Warrior).

Internally: Heal.execute(Warrior, Warrior) is called → heal amount is calculated → Warrior.heal(100) is called.

Then the Potion count is reduced through Item.changeCount(-1).

Result: Warrior's HP is restored, and the Potion count decreases.

Step 7 - Goblin is stunned again and skips.

Similar to Step 5, skips Goblin's turn again. For EndTurn: the Stun effect duration is reduced further. If the duration reaches zero, the effect is removed.

Step 8 – Warrior's normal attack eliminates Goblin.

External Trigger: Warrior's turn begins.

TriggerEffect: Warrior resets temporary turn state and activates any current status effects.

ChooseAction: The player selects Basic Attack.

ChooseTarget: The player selects Goblin as the target.

ApplyAction: The selected Action is executed through currentAction.use(Goblin).

Internally: Damage.execute(Warrior, Goblin) is called → damage is calculated → Goblin.takeDamage(damage) is called.

Result: Goblin's HP drops to 0.

After action application: The engine checks whether the target is still alive. Since Goblin is defeated, the engine logs the defeat and removes the enemy from the active enemy list.

EndTurn: Warrior finishes its turn. If no enemies remain, the battle ends.

Summary

This sequence diagram illustrates a combat interaction between a Warrior and a Goblin.

1. The process begins at the CLI (boundary), where the player inputs a command.
2. Then the input is passed to the BattleEngine.
3. The BattleEngine determines whose turn it is based on selected turn order.
4. At the start of each turn, current combatant resets any temporary turn state and processes active status effects.
5. The player then selects an action, for example, Basic Attack, Shield Bash, or Use Potion.
6. Then an Action object is selected, with corresponding ActionLogic.
7. If needed, a target is chosen before the action is applied.
8. The Action executes through its configured combat mechanisms, such as damage, healing, or effect application. Target's HP is updated.

9. If a skill is used, cooldown is triggered, and status effects such as stun are applied.
10. The BattleEngine updates status effect durations, skill cooldowns, and checks win/loss conditions, including whether backup enemies need to be spawned.
11. The cycle repeats until the game ends.

Input & Validation

- The process begins at the CLI, where the player inputs a command (e.g., BasicAttack, Shield Bash, or Use Potion).
- The input is passed to the BattleEngine.
- Validation: Before proceeding, the Engine verifies whether the selected action is valid and if a target is required.

Turn Resolution

The BattleEngine determines whose turn it is using a configured turn-order strategy. For each turn, the system processes the combat phases in sequence:

1. TriggerEffect
2. ChooseAction
3. ChooseTarget
4. ApplyAction
5. EndTurn

If conditions for entering a phase are not met (checked by calling the phase's canEnter() method), the phase is skipped. This allows us to skip phases, which is helpful in situations such as when the combatant in turn has the Stun applied, and hence needs to skip the middle phases and go directly from TriggerEffect phase to EndTurn phase.

8. Additional Features

- We created a new item named Elixir, which adds Stun Effect to the user's Basic Attack. For example, if a combatant uses the Elixir and performs a basic attack, the target would receive damage AND become stunned. To do so, we added a new class that implements the CombatMechanism interface, named EnhanceAction (used to add a mechanism to an action object), and added it to Elixir's mechanismList. During this process, we did not have to touch upon other existing classes such as Combatant, BattleEngine, or those implementing the CombatMechanism interface . This not only demonstrates key design principles such as OCP, but also highlights extensibility and the benefit of using objects to represent mechanisms instead of hardcoding the logic of each item or skill. For example, if we had hardcoded the logic for each item and skill, we would not be able to modify the behaviors of such objects during runtime. However, because we implemented item and skill logic through collections of separate mechanism objects, we were able to modify the behavior by adding a new object.
- We also created a new logic for determining the turn order using the class HpBasedOrder. The default logic was to use combatant speed to determine turn order, but we were able to create this new class to add a logic for determining turn order based on remaining HP (lower HP acts first). Because it implement the pre existing TurnDecider interface and realized its abstract methods, we did not have to modify any other class to make this new component to the game.

9. Reflection (Group Reflection)

This project was completed as a team of 4, with each member responsible for a different component of the system. One challenge would be coordinating and delegating the different taskings; some taskings require others to be completed first. To solve this, we set weekly team meetings, using them as deadlines. This creates a consistent time and place for us to share what we have completed, and to solicit feedback, opinions, and assistance if need be from each other. This improved our collaboration and communication to ensure clear alignment and integration of our work.

As the project grew, the complexity increased as the number of variables, methods, and functions increased. From this, we realised the importance of a UML Class Diagram, and on keeping it up to date. It was often used to help visualise the structure of the program clearly, and provides an easy reference for us to keep track of each class, helping with traceability when debugging. Overall, this helped us to save time when adjustments were needed.

This project also impressed upon us the importance of modular design, and the practical application of SOLID design principles. To explain this, one of our earlier discussions was deciding where the actions logic and functions should reside. Many of them were initially planned to reside in Combatant class (e.g. turn counter for buffs). This led to a cluttered code, especially within Combatant, and made it difficult to navigate through. We quickly understood this was not the best coding practice. So to resolve this, we applied the Single Responsibility Principle to split the code into multiple separate models and controllers, where each of them handled more specific tasks. This not only promoted better cohesion, maintenance, and readability of the code, but also highlighted the importance of good software design and practice.

Overall, we learnt that building an OOP application is not as straightaway compared to some of the taskings in our lectures and tutorial. It involves many small details that often fit like puzzle pieces, and this requires strong consistency; from how we design to implementation. This project strengthened our technical and teamwork skills, encouraging us to overcome challenges through online and offline collaboration, and through iterative improvements on our design. The final system is modular, extensible, and well-aligned with OOP design principles, providing a strong foundation for future enhancements or modifications. And we aim to continue cultivating and applying these good habits for our future programming modules.

Appendix B

<u>Declaration of Original Work for SC2002/CE2002/CZ2002 Assignment</u>

We hereby declare that the attached group assignment has been researched, undertaken, completed, and submitted as a collective effort by the group members listed below.

We have honored the principles of academic integrity and have upheld Student Code of Academic Conduct in the completion of this work.

We understand that if plagiarism is found in the assignment, then lower marks or no marks will be awarded for the assessed work. In addition, disciplinary actions may be taken.

Name	Course (SC2002/CE2002 CZ2002)	Lab Group	Signature /Date
XU ZIHE	SC2002	FCS1 (Group 4)	Xu Zihe 180426
YAP YI SHENG	SC2002	FCS1 (Group 4)	Yisheng 180426
YEO QUAN JIE	SC2002	FCS1 (Group 4)	180426
YOON DOYOUNG	SC2002	FCS1 (Group 4)	180426

Appendix C:

Declaration on Use of GAI (Generative Artificial Intelligence) Assistance in relation to Assignment/Project (to be submitted individually even for group projects)

I _____ (student name), _____@e.ntu.edu.sg (NTU email) honestly and sincerely make the following declaration in relation to the following course submission:

1. Name of course:

2. Course Code:

3. Instructor:

4. Title of Assignment/Project Submission:

In relation to the foregoing I hereby declare that, fully and properly in accordance with the Assignment/Project Instructions I have (check where appropriate):

i. Used GAI as permitted to assist in generating key ideas only. ☐

ii. Used GAI as permitted to assist in generating a first text only. ☐

And/or

iii. Used GAI to refine syntax and grammar for correct language submission only.

☐

Or

iv. As it is not permitted: Not used GAI assistance in any way in the development or generation of this assignment or project.

☐

I also declare that I have :

a. Fully and honestly submitted the digital paper trail required under the assignment/project instructions; and that

b. Wherever GAI assistance has been employed in the submission in word or paraphrase or inclusion of a significant idea or fact suggested by the GAI assistant, I have acknowledged this by a footnote; and that,

c. Apart from the foregoing notices, the submission is wholly my own work.

_____

_____ Student Name & Signature Date

(Optional) Sample Reflection Questions:

In 150 words or less, briefly state up to 5 key ideas that you formulated that the GAI tools alone could not have generated.